HTTP Status Codes in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

HTTP Status Code in ASP.NET Core Web API

In this article, I will discuss **HTTP Status Codes in ASP.NET Core Web API**. Returning the response with a proper status code is the backbone of any restful Web API. Now, it is time to learn how to format the response with the proper response status code as per our business requirements.

What are HTTP Status Codes?

HTTP status codes are an essential part of any Web API, i.e., Restful Services, as they provide information about the status of the HTTP request. In ASP.NET Core Web API, these status codes can be returned to the client (such as a browser or a mobile app) from the server to indicate success, failure, or the state of the requested resource.

What are the Different Categories of HTTP Status Codes?

HTTP status codes are grouped into five categories, each defined by the first digit of the code. Here, XX will represent the actual number.

1xx (Informational) Status Codes:

These Status Codes inform the client that the request has been received and that the server is continuing to process it. This category is typically used for interim responses and is not widely used in most client-server interactions. For example, 100 Continue means that the initial part of a request has been received and has not yet been rejected by the server. The following are some of the commonly used Status Codes in this Category:

- **100 Continue:** This informs the client that the initial part of the request has been received and that the client should continue sending the rest of the request.
- **101 Switching Protocols:** Indicates that the server is switching to a different protocol as requested by the client.

2xx (Successful) Status Codes:

This category of status codes indicates that the client's request was successfully received, understood, and accepted by the server. It confirms that the action the client intended to perform was completed. That means these status codes are used to signify a successful interaction, whether it's retrieving a resource (200 OK), creating a new resource (201 Created), or performing a request that does not need to return data (204 No Content). The following are some of the commonly used Status Codes in this Category:

- **200 OK:** The HTTP 200 OK success status response code indicates that the request has succeeded. It is often used for GET and POST requests that are processed successfully.
- **201 Created:** The request has succeeded, and a new resource has been created as a result. This is typically used in response to a POST request.
- **202 Accepted:** The server has received the request and will process it, but the processing may not be completed immediately.
- **204 No Content:** The server successfully processed the request but is not returning any content. This is often used for DELETE requests.

3xx (Redirection) Status Codes:

These Status codes tell the client that further actions need to be taken in order to complete the request. This usually involves the client making a subsequent request to a different URL. For example, 301 Moved Permanently indicates that the resource requested has permanently moved to a new location, and the new URL is provided in the Location Header of the response. These are important for maintaining old links that have moved to new URLs and directing clients to the correct resources. The following are some of the commonly used Status Codes in this Category:

- **301 Moved Permanently:** This response status code tells the client that the resource has been moved permanently to a new URL and should use the new URL in future requests.
- **302 Found:** This response status code indicates that the resource is temporarily located at a different URL. The client should use this temporary URL for this request. As the redirection might be altered occasionally, the client should continue using the original URI for future requests.
- **304 Not Modified:** This response status code informs the client that the resource has not changed since the last request so that the client can use its cached version.

4xx (Client Error) Status Codes:

This group of Status Codes indicates that there was an error on the client's side, meaning the request contains bad syntax, invalid data, or a missing request header. Whenever we get 4XX as the response code, it means there is some problem with our request.

For example, 404 Not Found is one of the most recognizable codes, indicating that the server can't find the requested resource. Another example is 400 Bad Request, meaning the server cannot process the request due to invalid request data. 401 is Unauthorized, which means the client has provided invalid authentication credentials. The 403 HTTP Status code means that authentication is successful, but the user is not authorized. These statuses are important for notifying clients that they need to modify their requests before resubmitting. The following are some of the commonly used Status Codes in this Category:

- **400 Bad Request:** The server cannot process the request due to a client error (e.g., malformed request syntax).

- **401 Unauthorized:** Authentication is required and has failed or has not been provided.
- **403 Forbidden:** The server understands the request but refuses to authorize it due to lack of permission.
- **404 Not Found:** The server can not find the requested resource. In the browser, this means the URL is not recognized. In an API, this can also mean that the endpoint is valid, but the resource itself does not exist.
- **405 Method Not Allowed:** This indicates that the server knows the request method but is not supported by the target resource. For example, we have one method, which is a POST method, in the server, and we are trying to access that method from the client using GET Verb; then, in that case, we will get a 405-status code.

5xx (Server Error) Status Codes:

These status codes indicate that the server failed to fulfill a valid request. Errors in this category are considered server-side errors. Whenever we get 5XX as the response code, it means there is some problem in the server. For example, 500 Internal Server Error suggests a generic error when an unexpected condition is encountered, and 503 Service Unavailable indicates that the server is not ready to handle the request, often due to maintenance or overload. This indicates that the problem is on the server's end and not due to the client's request. The following are some of the commonly used Status Codes in this Category:

- **500 Internal Server Error:** The server encountered an unexpected condition that prevented it from fulfilling the request.
- **501 Not Implemented:** The server either does not recognize the request method or is unable to fulfill the request.
- **502 Bad Gateway:** The server received an invalid response from the upstream server it accessed while trying to fulfill the request.
- **503 Service Unavailable:** The server is not ready to handle the request. Common causes include a server that is down for maintenance or overloaded.
- **504 Gateway Timeout:** This status code indicates that the server, while acting as a gateway or proxy, did not get a response from the upstream server in time to complete the request.

Note: In ASP.NET Core Web API, these status codes can be returned explicitly by using the StatusCode method in a controller action or by returning a specific result type like Ok(), NotFound(), Created(), Accepted(), BadRequest(), etc., which encapsulates these HTTP status codes. In our next article, we will discuss how to return these status codes from the controller action method.

Why HTTP Status Codes are Important?

If we want to consume any Restful API, we will send an HTTP Request, and in return, we will get the response, which includes response data, response headers, and an HTTP Status code. The HTTP Status code is important because it tells the client (the client who initiates the request, for example, Web, Android, iOS, Postman, IoT, Fiddler, etc.) what exactly happened to the request. If you send the wrong HTTP Status code, the client, i.e., the API consumer, will be confused.

So, HTTP status codes play an important role in effective communication between a client (Web, Android, iOS, Postman, IoT, Fiddler, etc.) and a server. Here's why they are important:

- **Communication Clarity:** Status codes provide essential feedback to clients about the results of their requests. They convey whether a request was successful, if there were any issues, or if further action is needed. For example, a 200 OK status code immediately informs the client that the request was successful, and the expected data is included in the response.
- **Efficient Troubleshooting:** Status codes help identify and diagnose issues with requests. For example, a 404 Not Found status code tells the developer that the requested resource does not exist on the server, which can prompt a check for correct URLs or resource availability, while a 500 Internal Server Error suggests a problem with the server.
- **User Experience:** Proper use of status codes can enhance user experience by providing meaningful feedback and appropriate responses to different scenarios. For example, a 401 Unauthorized status code can trigger the client to prompt the user for authentication details, thereby guiding the user towards the next step to access the resource. Similarly, a 403 Forbidden message informs users they don't have permission to access a resource, while a 200 OK message confirms successful operations.
- **System Monitoring and Optimization:** Analyzing the frequency and types of status codes can help web administrators understand traffic patterns, troubleshoot issues, and optimize server performance. For example, a high number of 500 Internal Server Error codes could indicate systemic backend issues needing attention.
- **Security:** Certain status codes, like 401 Unauthorized or 403 Forbidden, help in managing access control and protecting resources. They prevent unauthorized access and provide information about authorization status.

Summary of HTTP Response Status Codes:

1. **1xx Informational:** The request was received and is continuing.
2. **2xx Successful:** The request was successfully received, understood, and accepted.
3. **3xx Redirection:** Further action is needed to complete the request.
4. **4xx Client Error:** There was an error in the request sent by the client.
5. **5xx Server Error:** The server failed to fulfill a valid request.

In our next article, we will discuss **how to return 200 HTTP Status Codes in ASP.NET Core Web API**. In this article, I will try to give an overview of the HTTP Status Codes in ASP.NET Core Web API.

DOT NET TUTORIALS

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.